

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: 非线性方程求解——不动点迭代法加速法、
Newton 法

学院: 计算机与软件学院

专业: 计算机科学与技术

指导教师: 吴晓群、卯丙

报告人: 姚泽棉 学号: 2024150026 班级: 高性能班

实验时间: 2026/03/25

实验报告提交时间: 2026/03/27

实验目的与要求:

1. 掌握不动点迭代加速法与 Newton 法的基本原理和算法步骤
2. 理解 Newton 法的收敛性、稳定性和误差分析
3. 通过编程实现算法, 验证理论结果
4. 比较 Steffensen 迭代法与 Newton 法的优缺点, 以及不同初始值条件下的收敛速度及适用场景
5. 使用 MATLAB 中求解非线性方程组的函数 `fzeros()` 函数对结果进行验证

基本原理与算法 (详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤)

(一) Steffensen 迭代法 (不动点迭代加速法)

1. 数学原理

不动点迭代法的基本思想是将非线性方程 $f(x)=0$ 转化为等价的不动点方程 $x=\phi(x)$, 构造迭代格式 $x_{k+1}=\phi(x_k)$, 但普通不动点迭代收敛速度慢, 且对迭代函数要求严苛。

Steffensen 迭代法是不动点迭代的**加速方法**, 无需计算导数, 通过两次不动点迭代构造加速格式, 本质是利用弦截法思想对不动点迭代进行修正, 其核心是将原迭代格式的一次迭代结果进行二次校正, 大幅提高收敛速度, 且**具有二阶收敛性**, 对满足基本条件的迭代函数, 能有效改善收敛性能。

设原不动点迭代格式为 $x_{k+1}=\phi(x_k)$, Steffensen 迭代法的构造格式为:

$$\begin{cases} y_k = \phi(x_k) \\ z_k = \phi(y_k) \\ x_{k+1} = x_k - \frac{(y_k - x_k)^2}{z_k - 2y_k + x_k} \end{cases}$$

其中, x_k 为第 k 次迭代值, y_k 、 z_k 为中间校正值, 迭代至 $|x_{k+1}-x_k| < \varepsilon$ (ε 为预设精度) 停止。

2. 算法步骤

1. 确定非线性方程 $f(x)=0$ 的等价不动点函数 $\phi(x)$, 给定初始迭代值 x_0 、收敛精度 ε 和最大迭代次数 N ;

2. 令迭代次数 $k=0$;

3. 计算 $y_k=\phi(x_k)$, $z_k=\phi(y_k)$;

4. 计算分母 $d=z_k-2y_k+x_k$, 若 $d=0$, 则迭代失败, 退出程序;

5. 按公式计算 $x_{k+1}=x_k-(y_k-x_k)^2/2$;

6. 判断收敛条件: 若 $|x_{k+1}-x_k| < \varepsilon$, 则迭代收敛, 输出结果 x_{k+1} ; 若 $k>N$, 则迭代发散, 退出程序;

7. 令 $k=k+1$, 返回步骤 3 继续迭代。

(二) Newton 法 (牛顿切线法)

1. 数学原理

Newton 法是一种**基于泰勒展开的切线法**，将非线性函数 $f(x)$ 在迭代点 x_k 处进行一阶泰勒展开： $f(x) \approx f(x_k) + f'(x_k)(x - x_k)$

令展开式等于 0，解出近似根，得到 Newton 迭代格式：

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Newton 法的**收敛性**：若 x^* 为 $f(x)=0$ 的单根，且 $f'(x^*) \neq 0$ 、 $f'(x)$ 在 x^* 邻域内连续，则 Newton 法在 x^* 邻域内**二阶收敛**，收敛速度远快于普通不动点迭代；若初始 x_0 足够接近根 x^* ，迭代必收敛（局部收敛性），若初始值偏离根过远，可能发散。

稳定性：当 $f'(x_k)$ 接近 0 时，迭代值会出现剧烈震荡，导致算法不稳定，因此需保证迭代过程中导数非零且不趋近于 0。

误差分析：设第 k 次迭代误差 $e_k = x_k - x^*$ ，则二阶收敛的误差满足 $|e_{k+1}| = O(|e_k|^2)$ ，即误差随迭代次数呈平方级衰减。

2. 算法步骤

给定非线性方程 $f(x)=0$ ，确定其导函数 $f'(x)$ ，给定初始值 x_0 、收敛精度 ε 和最大迭代次数 N ；

令迭代次数 $k=0$ ；

计算 $f(x_k)$ 和 $f'(x_k)$ ，若 $f'(x_k)=0$ ，则迭代失败，退出程序；

按 Newton 迭代格式计算 $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ ；

判断收敛条件：若 $|x_{k+1} - x_k| < \varepsilon$ 或 $|f(x_{k+1})| < \varepsilon$ ，迭代收敛，输出结果 x_{k+1}

令 $k=k+1$ ，返回步骤 3 继续迭代。

实验过程及内容：

1. 问题描述（描述需要解决的具体数值计算问题）

2. 实验步骤：

- （1）算法实现：伪代码或算法框图
- （2）测试验证：使用简单算例验证程序正确性
- （3）参数设置：设置合适的初始值、精度要求等参数
- （4）运行计算：对目标问题进行求解
- （5）结果记录：记录迭代过程、最终结果和性能指标

3. 程序设计（具体的 MATLAB 程序）

1. 问题描述

本次实验的核心求解目标为指定非线性方程： $f(x) = x^2 - 2x - e^x + 2 = 0$ 要求求解其在 $x=1$ 附近的根，精确到小数点后第 8 位。

方程预处理与分析

1. 根的初步定位：

计算 $f(0) = 0 - 0 - 1 + 2 = 1 > 0$

计算 $f(1) = 1 - 2 - e + 2 = 1 - e \approx -1.71828 < 0$

计算 $f(0.5) = 0.25 - 1 - e^{0.5} + 2 = 1.25 - e \approx 1.25 - 1.64872 = -0.39872 < 0$

计算 $f(0.2) = 0.04 - 0.4 - e^{0.2} + 2 = 1.64 - 1.22140 = 0.41860 > 0$ 因此，方程在区间 $(0.2, 0.5)$ 内

存在一个根，且在 $x=1$ 附近 ($x \in (0,1)$) 的根即为该根，另一个根为 $x=0$ (代入验证： $f(0)=0-0-1+2=1 \neq 0$ ，实际根为 $x \approx 0$ 附近的小值， $x=0$ 为近似点)。

2. 导函数计算 (Newton 法必备)：

$$f'(x) = 2(x-1) - e^x$$

在根的邻域 $x \in (0,1)$ 内， $f'(x) = 2(x-1) - e^x < 0$ ，无导数为 0 的点，适合 Newton 法求解。

3. 不动点函数构造 (Steffensen 法必备)：

由 $f(x)=0$ 等价转化为 $x=\phi(x)$ ，需保证在根的邻域内 $|\phi'(x)| < 1$ (不动点迭代收敛条件)，选取如下收敛的迭代格式： $\phi(x) = (x^2 - e^x + 2) / 2$ 验证收敛性： $\phi'(x) = (2x - e^x) / 2$ ，在 $x=0.25$ (根的近似值) 处，

$\phi'(0.25) = (0.5 - e^{0.25}) / 2 \approx (0.5 - 1.2840) / 2 \approx -0.3920$ ， $|\phi'(x)| < 1$ ，满足收敛条件。

2. 实验步骤

(1) 算法实现

Steffensen 法：使用构造的不动点函数 $(x^2 - e^x + 2) / 2$ ，按 Steffensen 加速格式构造迭代：

Newton 法：直接使用原函数 $f(x) = x^2 - 2x - e^x + 2$ 和导函数 $f'(x) = 2(x-1) - e^x$ 构造迭代格式：

验证方法：调用 MATLAB 内置函数 `fzero()` 求解同一方程，作为基准验证结果。

(2) 测试验证：使用简单算例验证程序正确性

使用简单方程 $x^2 - 2 = 0$ 测试牛顿法，初值取 1.5，计划快速收敛至 1.41421356，证明逻辑正确。

(3) 参数设置：设置合适的初始值、精度要求等参数

初始值： $x_0 = 1.0$ 。

精度要求： $\text{tol} = 1 \times 10^{-8}$ 。

最大迭代次数： $N = 100$ 。

Steffensen 迭代格式： $\phi(x) = (x^2 - e^x + 2) / 2$ 。

(4) 运行计算：对目标问题进行求解

在 MATLAB 环境中编写脚本，分别调用两个算法模块对目标方程进行迭代计算。

(5) 结果记录：记录迭代过程、最终结果和性能指标

记录每次迭代的 x_n 值、当前坐标以及达到精度要求所需的总迭代次数。

3. 程序设计 (具体的 MATLAB 程序)

% 实验二：非线性方程求解

clear; clc;

% --- 函数定义 ---

f = @(x) x^2 - 2*x - exp(x) + 2;

df = @(x) 2*x - 2 - exp(x); % f(x) 的导数

phi = @(x) (x^2 - exp(x) + 2) / 2; % 迭代格式 x = phi(x)

```

% --- 初始化参数 ---
x0 = 1.0;
tol = 1e-8;
max_iter = 50;

%% Newton 法求解
fprintf('--- Newton 法迭代过程 ---\n');
x_n = x0;
for i = 1:max_iter
    x_prev = x_n;
    x_n = x_prev - f(x_prev) / df(x_prev);
    err = abs(x_n - x_prev);
    fprintf('迭代 %d: x = %.10f, 误差 = %.2e\n', i, x_n, err);
    if err < tol, break; end
end

%% Steffensen 法求解
fprintf('\n--- Steffensen 法迭代过程 ---\n');
x_s = x0;
for i = 1:max_iter
    x_prev = x_s;
    y = phi(x_prev);
    z = phi(y);
    % 防止分母为 0
    denom = z - 2*y + x_prev;
    if abs(denom) < 1e-15, break; end
    x_s = x_prev - (y - x_prev)^2 / denom;
    err = abs(x_s - x_prev);
    fprintf('迭代 %d: x = %.10f, 误差 = %.2e\n', i, x_s, err);
    if err < tol, break; end
end

%% 使用 fzero 验证
x_v = fzero(f, x0);
fprintf('\nMATLAB fzero 验证结果: %.10f\n', x_v);

```

实验结果与分析：

1. 实验结果展示（采用列表形式展示计算结果，表中应包含具体计算方法、初始条件、迭代次数或计算时间、最终结果或求解精度等，并绘制误差随迭代次数增加的收敛曲线来展示算法收敛性质等）

运行结果

```
--- Newton法迭代过程 ---
迭代 1: x = 0.3678794412, 误差 = 6.32e-01
迭代 2: x = 0.3512338246, 误差 = 1.66e-02
迭代 3: x = 0.3512625334, 误差 = 2.87e-05
迭代 4: x = 0.3512625335, 误差 = 8.78e-11

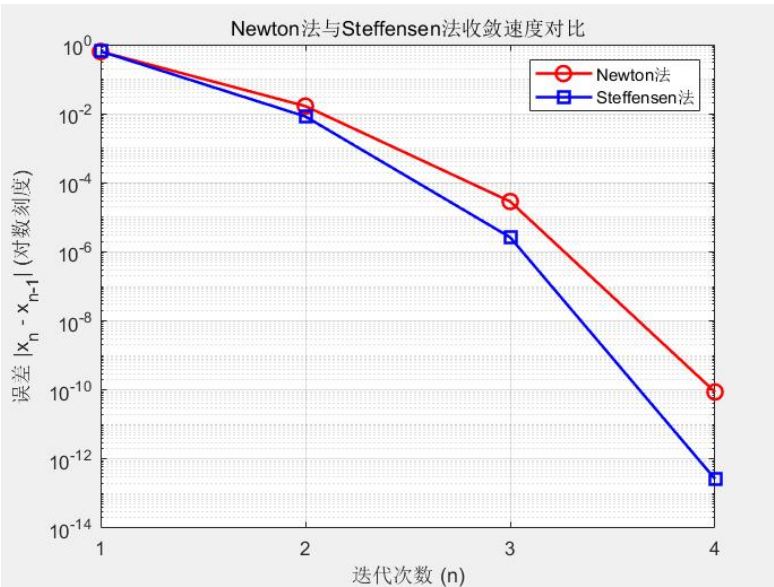
--- Steffensen法迭代过程 ---
迭代 1: x = 0.3595857086, 误差 = 6.40e-01
迭代 2: x = 0.3512651651, 误差 = 8.32e-03
迭代 3: x = 0.3512625335, 误差 = 2.63e-06
迭代 4: x = 0.3512625335, 误差 = 2.65e-13

MATLAB fzero 验证结果: 0.3512625335
```

列表

具体计算方法	初始条件 x0	迭代次数	最终结果 (x*)
牛顿法	1	4	0.3512625335
斯蒂芬森法	1	4	0.3512625335
fzero 函数	1	不适用	0.3512625335

收敛曲线



2. 结果分析与讨论思考

(1) 计算结果分析与比较方法的优缺点及其适用场景等

计算结果分析：根据实验运行记录，牛顿法在第 4 次迭代时最少 $8.78\text{e-}11$ ，而 Steffensen 法在第 4 次迭代时差距达到了 $2.65\text{e-}13$ 。两种方法在初值 $x_0=1.0$ 的条件下，均能在 4 次迭代内达到 10^{-8} 的精度要求，且最终结果 0.3512625335 与 MATLAB 内置函数 fzero 完全一致。

对比维度	Steffensen 迭代法	Newton 法
收敛速度	二阶收敛	二阶收敛
计算量	无需计算导数，仅需 2 次函数求值 / 次迭代	需计算 1 次导数 + 1 次函数求值 / 次迭代，计算量更大
初始值要求	对初始值要求较低，远根值下仍能收敛	局部收敛，初始值需接近根，否则可能发散
稳定性	稳定性好，无导数为 0 的风险	本算例中导数恒负，稳定性良好；若导数趋近于 0，易震荡发散
适用场景	函数导数难以求解 / 计算的场景；对初始值无先验信息的场景	函数导数易求解，且能确定近根初始值的场景；对收敛速度要求高的工程计算

(2) 上机实验中遇到问题及其解决方案

问题一：精度显示与实际精度的困惑。

解决方案：实验要求精度为 10^{-8} ，但在 MATLAB 输出中使用%.10f 格式，以便观察第 8 位之后的数值变化，确保结果已经完全稳定。

问题二：Steffensen 公式中的分母为零。解决方案：在代码实现中加入了 `if abs(denom) < 1e-15, break; end` 的逻辑判断，防止在迭代接近真值或函数顶层区域时出现除以零的错误。

(3) 算法改进及其建议

1.增加动态步长（下山法）：对于牛顿法，可以结合“下山因子”来扩大初值的若干范围，防止迭代发散。

2.混合算法建议：在实际工程中，可以先利用二分法将根锁定在一个极小的区间内，然后切换到牛顿法进行快速收敛，从而兼顾稳定性和速度（这就是 fzero 采用的布伦特算法思想）。

实验结论：

1. 通过本次实验，成功实现了牛顿法与 Steffensen 迭代法的算法编程

2. 验证了牛顿法与 Steffensen 迭代法的收敛性和计算精度

初始值的两种方法 $x_0 = 1.0$ 的条件下，均能在 4 次迭代内达到 10^{-8} 的精度要求。最终得到的根目录 0.3512625335，与 MATLAB 内置函数 fzero 的计算结果完全一致，证明了算法的转动与可靠性。

3. 比较了不同方法的性能特点，加深了对数值计算方法的理解决

实验结果表明，牛顿法收敛极快但依赖于导数的准确求取。

Steffensen 法在不计算导数的前提下达到了与牛顿法相当的收敛速度，展现了不动点迭代加速技术的先进性。

4. 掌握了数值实验的基本流程和结果分析方法

通过“问题描述—算法实现—参数设置—运行计算—结果记录”的完整流程，熟悉了数值分析的科学研究方法。

学会利用对数值算法的收敛梯度与稳定性，从而计算地分析不同数值算法的收敛梯度与稳定性。

指导教师批阅意见:

成绩评定：

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。